

UNIVERSIDADE FEDERAL DE ALFENAS – UNIFAL-MG

INSTITUTO DE CIÊNCIAS EXATAS – ICE_x

BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO

**RELATÓRIO TÉCNICO:
IMPLEMENTAÇÃO DO PROTOCOLO
GO-BACK-N EM JAVA VIA UDP**

Alunos: Luís Felipe Barbosa Leite e Miguel Leone

Professor: Flavio Barbieri Gonzaga

Alfenas, MG

2026

1 Introdução e Fundamentação Teórica

O protocolo UDP (*User Datagram Protocol*) foi escolhido como base para a comunicação na camada de transporte devido à sua natureza não orientada a conexão e à ausência de garantias de entrega, sequenciamento ou controle de fluxo. Para estabelecer uma transferência confiável de dados sobre esse canal inerentemente não confiável, implementou-se a máquina de estados finita (FSM) do protocolo *Go-Back-N* (GBN), conforme descrito pela literatura clássica de Kurose e Ross. Essa abordagem transfere para a camada de aplicação a complexa responsabilidade de gerenciar perdas, corrupções de bits e a reordenação de pacotes, garantindo a integridade final da transferência.

2 Decisões de Projeto e Arquitetura

A implementação foi dividida em três entidades principais: **Emissor**, **Receptor** e **Pacote**. O sistema foi desenvolvido estritamente em Java, utilizando a API de Sockets UDP (`DatagramSocket` e `DatagramPacket`).

- **Estrutura do Datagrama e Verificação de Erros:** O datagrama foi desenhado para maximizar a eficiência de envio, limitando o *payload* a 1024 bytes úteis. O cabeçalho personalizado de 15 bytes carrega os metadados fundamentais da FSM: o identificador do tipo de pacote (DADOS, ACK, HANDSHAKE ou FIN), o número de sequência, o número de confirmação e o tamanho exato do *payload* inserido. Para mitigar o risco de corrupção de bits durante o trânsito na rede física, implementou-se um algoritmo de Checksum aditivo. O valor de verificação é calculado no emissor somando-se os bytes individuais do cabeçalho e da carga útil, e o resultado é validado na recepção. Pacotes que apresentam falha de integridade matemática são tratados pela FSM do receptor como perdas absolutas, sendo descartados silenciosamente para forçar a retransmissão por *timeout*.
- **Controle de Fluxo e Temporização:** A máquina de estados do emissor foi implementada utilizando um *buffer* circular (`Pacote[] bufferEnvio`) para gerenciar a janela deslizante de tamanho N . O bloqueio do envio, quando a janela enche, é controlado pela sincronização do objeto e pelo uso de `wait()` e `notifyAll()`. A temporização foi resolvida com as classes nativas `Timer` e `TimerTask`, garantindo

que o estouro de tempo limite acione a retransmissão de todos os pacotes pendentes a partir do pacote não confirmado mais antigo (base).

- **Simulação de Perdas e Confiabilidade:** A máquina de estados do receptor descarta pacotes fora de ordem e reenvia o ACK do último pacote recebido corretamente. A simulação de perdas foi implementada avaliando a geração de um número aleatório em relação à probabilidade configurada; caso o teste indique perda, o pacote é descartado silenciosamente na camada de aplicação sem o envio do ACK correspondente. Adicionalmente, foi incluída a geração e verificação de hash MD5 (`MessageDigest`), cuja assinatura final é transferida no pacote de encerramento (`FIN`) para atestar a integridade completa do arquivo binário transferido.

3 Dificuldades Encontradas

Durante o desenvolvimento, os principais desafios envolveram a concorrência e a sincronização no módulo Emissor. A arquitetura do emissor GBN exige um alto nível de assincronia: enquanto a *thread* principal realiza a leitura do arquivo em disco e dispara pacotes de dados para o *socket* UDP, uma *thread* secundária permanece bloqueada na escuta contínua de reconhecimentos (ACKs) cumulativos vindos do receptor.

O desafio central de engenharia de software consistiu no controle da janela deslizante, representada pelas variáveis base e próximo número de sequência. Para evitar condições de corrida ao atualizar os ponteiros da janela circular a partir de *threads* distintas, fez-se uso de blocos estritamente sincronizados. Quando a distância cíclica entre o pacote atual e a base atinge a capacidade máxima da janela configurada, a *thread* de transmissão é suspensa com segurança até que a recepção de um ACK libere espaço no *buffer*. Também foi necessário um cuidado redobrado para garantir que a aplicação não travasse aguardando o recebimento de pacotes finais perdidos durante o estado de encerramento da conexão (*Teardown*).

4 Testes Realizados e Análise de Resultados

Para validar a eficiência e o comportamento do protocolo sob diferentes condições, foram realizados testes de transferência de um arquivo binário de 1,57 MB (1.572.864 bytes).

Os experimentos foram divididos em duas análises principais.

4.1 Impacto do Tamanho da Janela (N)

Cenário: Transferência de arquivo com 10% de probabilidade de erro simulada no canal.

- **Janela $N = 4$:** Tempo total de 79,77s. Retransmissões: 632 pacotes.
- **Janela $N = 8$:** Tempo total de 95,62s. Retransmissões: 1.510 pacotes.
- **Janela $N = 16$:** Tempo total de 82,72s. Retransmissões: 2.624 pacotes.

Análise: O teste demonstra uma ineficiência inerente do protocolo *Go-Back-N* em redes instáveis. Como o receptor descarta qualquer pacote fora de ordem, a perda de um único datagrama no início de uma janela grande (como $N = 16$) condena todos os datagramas subsequentes, mesmo que tenham chegado intactos. Isso gerou uma “avalanche” no terminal: foram 2.624 pacotes retransmitidos e 2.460 descartados sumariamente por estarem fora de ordem. Em redes com alta taxa de falha, o uso de janelas longas desperdiça processamento e banda, justificando o uso de abordagens como o *Selective Repeat*.

4.2 Comportamento do Protocolo sob Probabilidade de Perdas

Cenário: Transferência com a Janela $N = 8$ fixada, variando a probabilidade teórica de falha.

Perda Teórica	Perda Efetiva	Total Enviado (incl. Retrans.)	Retransmissões	Tempo Total	Vazão Média
0%	0,00%	1.536 pacotes	0	0,40 s	31,46 Mbps
5%	5,42%	2.240 pacotes	704	44,54 s	0,28 Mbps
10%	11,01%	3.046 pacotes	1.510	95,62 s	0,13 Mbps
15%	14,24%	3.575 pacotes	2.039	128,35 s	0,10 Mbps
20%	20,70%	4.734 pacotes	3.198	201,52 s	0,06 Mbps

Tabela 1: Comparativo de desempenho do protocolo GBN sob variação de ruído estocástico.

Análise: Os dados atestam o funcionamento da Lei dos Grandes Números na simulação (a perda efetiva orbitou as métricas configuradas) e demonstram a degradação drástica da vazão da rede (*throughput*) proporcional ao aumento do ruído. Em um cenário ideal (0%), a transferência foi limitada apenas pelo hardware (0,40 segundos). Contudo, ao atingir 20% de instabilidade, o sistema congelou frequentemente aguardando os *timeouts*

do emissor, inflando o tempo de atividade para mais de 3 minutos e forçando mais retransmissões (3.198) do que o próprio tamanho original do arquivo (1.536 blocos). Em todos os cenários, o hash MD5 atestou “Sucesso Total”, validando que a máquina de estados superou o meio não confiável.

5 Conclusão

A implementação prática do protocolo *Go-Back-N* atestou com precisão os conceitos teóricos fundamentais das redes de computadores. O sistema demonstrou confiabilidade absoluta na entrega de arquivos binários, suportando com resiliência as perdas estocásticas simuladas pelo receptor e garantindo a integridade dos dados por meio da verificação final de hash MD5. Entretanto, a análise estatística do *throughput* sob taxas agressivas de erro revelou o grande gargalo desta implementação: a retransmissão não seletiva. Ao forçar o reenvio de toda a extensão de uma janela devido à perda de um único pacote, o GBN satura o canal de comunicação com tráfego redundante. Para cenários reais com alta probabilidade de erro, conclui-se que a evolução ideal para este sistema seria a transição para o protocolo *Selective Repeat* (SR), que otimizaria o uso da banda ao exigir a retransmissão exclusiva dos pacotes comprovadamente perdidos.